Lecture 30.11: Voting Classifier & Regressor (Implementation) — Scikit-Learn Pipeline







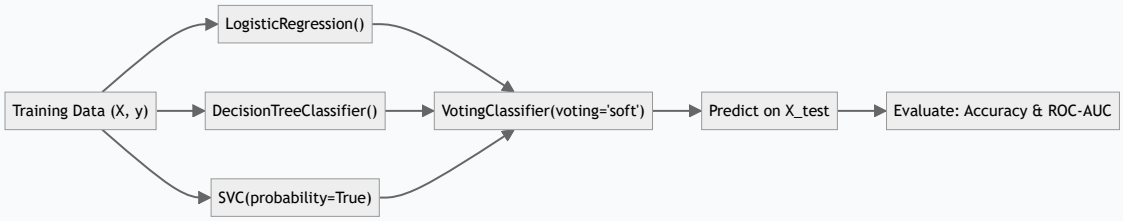




📌 Topic Overview

Production implementation of VotingClassifier and VotingRegressor in Scikit-Learn. Combines heterogeneous learners (Logistic Regression, Decision Trees, Support Vector Machines) with both hard and soft voting configurations. Evaluates metric improvements over standalone base models.

🗺️ Visual Architecture Diagram



🛠️ Mathematical Mechanics & Theoretical Foundations

Weighted Soft Voting prediction across base estimators with weights w_k:

$$\hat{y} = \arg\max_{c} \frac{\sum_{k=1}^{K} w_k P_k(y = c|\mathbf{x})}{\sum_{k=1}^{K} w_k}$$

💻 Python Production Implementation

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score

def train_heterogeneous_voting_classifier(random_state=42):
    """
    Builds, trains, and compares standalone models against Hard and Soft VotingCla
    """
    X, y = make_classification(
        n_samples=600, n_features=8, n_informative=5, random_state=random_state
    )
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=random_state, stratify=y
    )

    # 1. Base Estimators
    clf_lr = LogisticRegression(random_state=random_state)
    clf_dt = DecisionTreeClassifier(max_depth=4, random_state=random_state)
    clf_svc = SVC(probability=True, random_state=random_state)

    # Train standalone base models
    clf_lr.fit(X_train, y_train)
    clf_dt.fit(X_train, y_train)
    clf_svc.fit(X_train, y_train)

    acc_lr = accuracy_score(y_test, clf_lr.predict(X_test))
    acc_dt = accuracy_score(y_test, clf_dt.predict(X_test))
    acc_svc = accuracy_score(y_test, clf_svc.predict(X_test))

    # 2. Hard Voting Ensemble
    voting_hard = VotingClassifier(
        estimators=[('lr', clf_lr), ('dt', clf_dt), ('svc', clf_svc)],
        voting='hard'
    )
    voting_hard.fit(X_train, y_train)
    acc_hard = accuracy_score(y_test, voting_hard.predict(X_test))

    # 3. Soft Voting Ensemble
    voting_soft = VotingClassifier(
        estimators=[('lr', clf_lr), ('dt', clf_dt), ('svc', clf_svc)],
        voting='soft'
    )
    voting_soft.fit(X_train, y_train)
    acc_soft = accuracy_score(y_test, voting_soft.predict(X_test))
```

```
    results = {
        "LogisticRegression": acc_lr,
        "DecisionTree": acc_dt,
        "SVC": acc_svc,
        "Voting_Hard": acc_hard,
        "Voting_Soft": acc_soft
    }
    return results

# ----------------- Driver Execution & Testing -----------------
if __name__ == "__main__":
    benchmark = train_heterogeneous_voting_classifier()
    print("=" * 60)
    print("--- HETEROGENEOUS VOTING CLASSIFIER BENCHMARK ---")
    for model_name, acc in benchmark.items():
        print(f"  {model_name:22s} Test Accuracy: {acc * 100:.2f}%")
    print("=" * 60)
```

🔑 Critical Architectural Takeaways

1. **Multi-Family Generalization**: Combining diverse inductive biases (linear, tree-based, margin-based, metric-based) cancels out individual model blindspots.
2. **Leakage Protection via OOF**: Cross-validated out-of-fold predictions ensure the Level-1 meta-model optimizes strictly on unseen generalization behaviors.
3. **Calibrated Confidence Weights**: Soft voting and meta-learning outperform simple hard voting by incorporating prediction probability confidences.